
LogiFlow

Plataforma de ruteo dinámico de flotas en tiempo real

Documento de Arquitectura

Release 2 — Entrega Final

Los Gavilanes del Código

Andersson David Sánchez Méndez

Cristian Santiago Pedraza Rodríguez

Elizabeth Correa Suárez

Juan Sebastián Ortega Muñoz

Arquitecturas de Software — ARSW

Escuela Colombiana de Ingeniería Julio Garavito

Mayo 2026

Sistema en producción: logiflowapp.z13.web.core.windows.net

API Backend: logiflow-api.eastus2.cloudapp.azure.com

Repositorios: [Logiflow-Gavilanes-ECI/logiflow](https://github.com/Logiflow-Gavilanes-ECI/logiflow) |

[Logiflow-Gavilanes-ECI/logiflow-front](https://github.com/Logiflow-Gavilanes-ECI/logiflow-front)

Índice

1. Introducción	1
1.1. Objetivo	1
1.2. Alcance	1
1.3. Audiencia	1
2. Visión General	2
2.1. Descripción del sistema	2
2.2. Requerimientos funcionales clave	3
2.3. Requerimientos no funcionales clave	4
3. Marco Metodológico	4
3.1. Metodología	4
3.2. Criterios DoR / DoD	5
3.3. Planificación y estimación	5
4. Atributos de Calidad	6
4.1. Rendimiento	6
4.2. Escalabilidad	7
4.3. Seguridad	9
4.3.1. Escenario 1 — Autenticación en WebSocket	9
4.3.2. Escenario 2 — Autorización por rol	9
4.3.3. Escenario 3 — Confidencialidad en tránsito + refresh token rotation	9
4.4. Disponibilidad	10
4.4.1. SLA de componentes cloud (Azure)	11
4.4.2. SLA del software propio	11
4.4.3. Cuello de botella y plan de evolución a 3 nuevas	12
4.5. Portabilidad	12
4.6. Mantenibilidad	12
5. Vista de Arquitectura	14
5.1. Estilo arquitectónico	14
5.2. Modelo cloud	14
5.3. Decisiones arquitectónicas clave	15
6. Diagramas UML	16
6.1. Diagrama de Contexto (C4 Nivel 1)	16
6.2. Diagrama de Componentes (C4 Nivel 2-3)	17
6.3. Diagrama de Clases	19
6.4. Diagrama Entidad-Relación	20

6.5. Diagrama de Secuencia — Reoptimización por evento de tráfico	21
6.6. Diagrama de Despliegue	22
6.7. Diagrama de Actividades	24
7. Detalles Tecnológicos	25
7.1. Lenguajes y frameworks	25
7.2. Infraestructura y servicios	25
7.3. DevOps y CI/CD	26
7.3.1. Pipeline CI — Backend	26
7.3.2. Pipeline CD — Backend	27
7.3.3. Pipeline CI/CD — Frontend	27
8. Seguridad del Sistema	27
8.1. Autenticación y autorización	27
8.2. Cifrado	28
8.3. Buenas prácticas	29
9. Bitácora de Ceremonias	29
9.1. Sprint 0 — Iniciación (5 pts)	29
9.2. Release 1 (Sprints 1-5)	30
9.3. Release 2 (Sprints 6-10)	30
9.4. Planning, Review, Retrospectivas	31
9.5. Lecciones aprendidas	31
10. Conclusiones y Recomendaciones	32
10.1. Logros	32
10.2. Riesgos identificados	32
10.3. Mejora continua	33
A. Glosario	34
B. Referencias	35
C. Repositorios y enlaces	35

Índice de figuras

1.	Diagrama de Contexto — LogiFlow Backend Platform y sus actores externos	16
2.	Diagrama de Componentes — 5 servicios backend agrupados con sus componentes internos	17
3.	Diagrama de Clases UML — Controllers, Services, Repositories, DTOs, Entidades del Gateway	19
4.	Diagrama ER — Modelo PostgreSQL gestionado por Prisma	20
5.	Diagrama de Secuencia — Flujo end-to-end de reoptimización disparada por evento externo	21
6.	Diagrama de Despliegue — Azure Subscription, Resource Group, VM y Docker Compose stack	22
7.	Diagrama de Actividades — Procesamiento de evento de tráfico con swimlanes por responsable	24

Índice de tablas

1.	Requerimientos funcionales clave	3
2.	Requerimientos no funcionales clave	4
3.	Resumen de Sprints ejecutados — 245 story points entregados	6
4.	Resultados de pruebas de carga JMeter — 4 escenarios de 5 minutos cada uno	7
5.	Componentes escalables independientemente	8
6.	SLA garantizado por proveedor cloud — infraestructura Azure	11
7.	SLA estimado conservador para software propio	11
8.	Cobertura de tests — ambos repositorios	13
9.	Stack tecnológico por servicio	25

1. Introducción

1.1. Objetivo

Este documento describe la arquitectura de software de **LogiFlow**, una plataforma distribuida y cloud-native que resuelve el problema de ruteo dinámico de flotas (Dynamic Vehicle Routing Problem) en tiempo real. El objetivo principal es presentar las decisiones arquitectónicas, las vistas estáticas y dinámicas del sistema, las tácticas implementadas para atender los atributos de calidad exigidos en el curso (rendimiento, disponibilidad, seguridad, mantenibilidad, portabilidad) y la evidencia de cumplimiento medida sobre el sistema desplegado en producción.

El documento también busca servir como artefacto técnico de referencia para que cualquier desarrollador o evaluador externo pueda comprender, mantener, evolucionar o migrar el sistema sin acceso directo al equipo original.

1.2. Alcance

El alcance cubre la totalidad de los dos monorepos del proyecto:

- **Backend** ([Logiflow-Gavilanes-ECI/logiflow](#)) — nueve servicios contenedorizados: **gateway** (NestJS), **realtime** (Socket.io), **optimizer** (gRPC + VROOM), **ai-predictor** (Python/Flask), **vroom** (motor VRP), **n8n** (automatización), **openclaw** (notificaciones multicanal), **postgres** (base de datos) y **redis** (caché y broker pub/sub). En `docker-compose.yml`, **n8n** y **openclaw** están bajo el *profile automation*; en `docker-compose.prod.yml` corren siempre.
- **Frontend** ([Logiflow-Gavilanes-ECI/logiflow-front](#)) — dos aplicaciones Angular: **web-admin** (PWA para despachadores) y **mobile** (Ionic + Capacitor para conductores), y cuatro librerías TypeScript compartidas en `shared/`.
- **Infraestructura** — definida con Terraform sobre Azure (Máquina Virtual Linux + Storage Account Static Website + redes asociadas), con despliegue automatizado vía GitHub Actions.

Quedan fuera del alcance los siguientes elementos: integraciones futuras con sistemas ERP de terceros, módulos de facturación/cobro, motor de análisis predictivo a largo plazo basado en datos históricos persistidos en data lake, y soporte para dispositivos iOS (la aplicación móvil actual se entrega únicamente como APK Android).

1.3. Audiencia

Este documento está dirigido a:

- **Profesor y evaluadores** del curso de Arquitecturas de Software de la ECI, para verificar cumplimiento de la rúbrica de Release 2.
- **Desarrolladores que reciban el código** para mantenimiento o evolución futura.
- **Arquitectos de software** que necesiten replicar el patrón de ruteo dinámico en sistemas similares.
- **Stakeholders técnicos no codificadores** (jefes de operaciones logísticas) que requieran entender el sistema sin profundizar en el código.

2. Visión General

2.1. Descripción del sistema

LogiFlow es una plataforma de ruteo dinámico de flotas en tiempo real. Dado un conjunto de vehículos con capacidad y ubicación GPS, y un conjunto de paradas de entrega con demanda y ventanas de tiempo, el sistema resuelve el **Vehicle Routing Problem (VRP)** y entrega rutas óptimas a los conductores en sus dispositivos móviles.

Lo que diferencia a LogiFlow de un optimizador estático tradicional es que el sistema **recalcula la ruta óptima en menos de 4 segundos** ante cualquier evento externo: cierre de vía, accidente de tránsito, nueva orden urgente o avería mecánica de un vehículo. La ruta actualizada se entrega instantáneamente a todos los clientes conectados (despachadores en la web, conductores en la app móvil) vía WebSocket y Firebase Cloud Messaging.

Promesa de valor. *Reducir el tiempo de respuesta operacional ante eventos imprevistos de horas a segundos, minimizando entregas tardías y kilómetros recorridos innecesariamente.*

Contexto del problema. Según datos del sector logístico latinoamericano, el 23 % de las entregas llegan con retraso y el 40 % de los costos operativos de logística se generan por ineficiencias de ruteo evitables. LogiFlow ataca directamente este problema cerrando el ciclo entre detección de eventos, reoptimización y entrega de instrucciones al conductor.

2.2. Requerimientos funcionales clave

Tabla 1: Requerimientos funcionales clave

ID	Nombre	Descripción
RF-01	Autenticación Google OAuth	El sistema permite iniciar sesión únicamente con Google OAuth 2.0 (no usuario/contraseña local), con redirección diferenciada para admin/conductor según parámetro <code>?app=</code> .
RF-02	Gestión de flota	Operaciones CRUD sobre vehículos (id, placa, modelo, capacidad, estado, posición GPS) y paradas (dirección, coordenadas, demanda, prioridad).
RF-03	Ingesta de eventos	Webhook público que recibe eventos externos de tráfico/clima/incidentes a través de n8n, los normaliza, los enriquece con Google Maps y los enruta al backend autenticado.
RF-04	Optimización VRP	Cliente gRPC que invoca al <code>optimizer</code> , el cual prepara la matriz (Google Routes o request), opcionalmente la ajusta con AI Predictor según congestión en corredores de Bogotá, y resuelve con VROOM.
RF-05	Difusión en tiempo real	Emisión de eventos <code>route:update</code> , <code>vehicle:position</code> , <code>vehicle:status</code> , <code>stop:completed</code> vía Socket.io a rooms <code>fleet</code> y <code>vehicle:<id></code> .
RF-06	Notificaciones push	Envío de push notifications a conductores mediante Firebase Cloud Messaging cuando se actualiza su ruta.
RF-07	Alertas operacionales	Despacho multicanal de alertas (Telegram, Slack, Discord, Email) vía OpenClaw para eventos de riesgo alto/crítico.
RF-08	Visualización de flota	Dashboard web con mapa Google Maps en tiempo real, lista de vehículos con estado, registro de eventos y vista detallada de ruta por vehículo.
RF-09	Ruta del conductor	Pantalla móvil con mapa, lista ordenada de paradas, botones de estado (Start/Arrived/Delivered) y polyline conectando las paradas.
RF-10	Heartbeat y offline	Detección automática de vehículos sin señal mediante heartbeat de 15s, marcado como offline y notificación a la flota.

2.3. Requerimientos no funcionales clave

Tabla 2: Requerimientos no funcionales clave

ID	Atributo	Meta y justificación
RNF-01	Rendimiento (latencia)	Tiempo end-to-end desde evento entrante hasta ruta entregada al cliente menor a 4 segundos.
RNF-02	Rendimiento (throughput)	Soportar al menos 20 transacciones por segundo (TPS) sosteniblemente sobre el camino crítico.
RNF-03	Disponibilidad	Disponibilidad de infraestructura cloud $\geq 99.7\%$ (Bass et al., cálculo en serie).
RNF-04	Seguridad (auth)	Toda comunicación sensible cifrada con TLS 1.2/1.3; passwords con bcrypt; JWT en handshake WebSocket; refresh tokens single-use.
RNF-05	Mantenibilidad	Cobertura de tests $\geq 80\%$ en statements tanto en backend como en frontend; quality gate Verde en SonarCloud.
RNF-06	Portabilidad	Cero líneas de código deben cambiar para migrar de Azure a otro proveedor (AWS, GCP, On-Premise). Toda la infra dockerizada + IaC con Terraform.
RNF-07	Escalabilidad	La arquitectura debe permitir escalado horizontal del Real-time Service mediante Redis Pub/Sub adapter.
RNF-08	Trazabilidad	Propagar x-correlation-id a través de toda la cadena: webhook $\rightarrow$ Gateway $\rightarrow$ Optimizer gRPC $\rightarrow$ Socket.io $\rightarrow$ logs.

3. Marco Metodológico

3.1. Metodología

El equipo trabajó bajo el marco **Scrum** usando **Azure DevOps** como herramienta oficial de gestión del backlog y tablero Kanban. Cada Sprint tuvo una duración de dos semanas. Las ceremonias estandarizadas fueron:

- **Sprint Planning** (lunes de inicio de sprint) — selección de Historias de Usuario del Product Backlog al Sprint Backlog con estimación en story points (escala Fibonacci 1, 2, 3, 5, 8, 13).
- **Daily Standup** asíncrono vía Discord debido a horarios de clase distintos por integrante.
- **Sprint Review** (viernes de cierre) — demostración funcional con el sistema corriendo, no sólo código en repo.

- **Sprint Retrospective** — documento corto en formato *Mad/Sad/Glad* archivado por sprint.

GitHub se utilizó para los dos repositorios con *branch protection* sobre `main` y `develop`: ningún cambio entra a estas ramas sin pull request aprobado por al menos un par y con la pipeline CI en verde.

3.2. Criterios DoR / DoD

Definition of Ready (DoR). Una Historia de Usuario está lista para entrar al Sprint cuando cumple:

1. Tiene título en formato Como [rol] quiero [acción] para [beneficio].
2. Cuenta con criterios de aceptación enumerados (al menos 3, formato Given/When/Then cuando aplica).
3. Está estimada en story points con consenso del equipo.
4. Tiene asignada al menos una etiqueta de servicio (`gateway`, `realtime`, `optimizer`, `frontend`, etc.).
5. Las dependencias técnicas (por ejemplo, otra historia que debe completarse antes) están declaradas en Azure DevOps.

Definition of Done (DoD). Una Historia se considera DONE cuando cumple:

1. Código merged a `main` vía pull request aprobado.
2. Pipeline CI en verde: lint + tests + cobertura $\geq 80\%$ para servicios afectados.
3. Tests unitarios o de integración cubriendo los criterios de aceptación.
4. Documentación actualizada (README del servicio o swagger del endpoint REST cuando aplica).
5. Demo realizada en vivo durante el Sprint Review.
6. Si afecta a producción: desplegado a Azure VM y validado con un smoke test.

3.3. Planificación y estimación

La planificación inicial contemplaba **7 sprints**; sin embargo, la curva de aprendizaje en tecnologías nuevas para el equipo (gRPC, Terraform, Google OAuth, Firebase FCM, Socket.io con Redis Pub/Sub) llevó el proceso a **10 sprints reales**. No se considera un

fracaso de planeación sino evidencia de un compromiso con tecnologías que se aprendieron de cero durante el semestre.

Tabla 3: Resumen de Sprints ejecutados — 245 story points entregados

Sprint	Nombre	Puntos	Estado
S0	Iniciación y setup de repos	5	DONE
S1	MVP Conectado — esqueletos de servicios + integración básica	13	DONE
S2	APIs Reales — CRUD vehicles/stops + persistencia Prisma	21	DONE
S3	Optimizer Vivo — VROOM + gRPC integrados	21	DONE
S4	Interfaces + Auth — frontend inicial + JWT	34	DONE
S5	Cloud Deploy Azure (Corte 2) — Terraform + Nginx + TLS	46	DONE
S6	Observabilidad + Tests — cobertura 80%+	34	DONE
S7	Escala / Performance — pruebas JMeter	21	DONE
S8-S10	OAuth + FCM + Hardening (Corte 3)	55	DONE
Total entregado:		245 pts	

4. Atributos de Calidad

Esta sección documenta los escenarios de calidad implementados, las tácticas arquitectónicas aplicadas siguiendo el marco de *Bass, Clements & Kazman — Software Architecture in Practice* y las métricas reales medidas sobre el sistema desplegado.

4.1. Rendimiento

Escenario de calidad.

- **Fuente:** Sistema externo (n8n) o usuario interno (despachador).
- **Estímulo:** Llegada de un evento de tráfico válido al webhook público.
- **Artefacto:** Camino crítico Gateway → Optimizer gRPC → VROOM → Redis → Socket.io → cliente.
- **Ambiente:** Producción en Azure East US 2 con carga concurrente.
- **Respuesta:** El sistema completa la reoptimización y entrega la ruta actualizada a los clientes.
- **Métrica de respuesta:** P95 de latencia ≤ 4 segundos; TPS ≥ 20 .

Pruebas de carga — Apache JMeter 5.6.3. Se ejecutaron pruebas de carga contra el backend desplegado en producción real en Azure East US 2. El flujo probado es el camino crítico completo: login, extracción del JWT, y POST al webhook que dispara todo el pipeline.

Tabla 4: Resultados de pruebas de carga JMeter — 4 escenarios de 5 minutos cada uno

Usuarios concurrentes	TPS	P95 (ms)	% Error	Diagnóstico
10	19.91	674	0.00 %	Saludable
25	21.10	1,546	0.00 %	Saludable
50	20.98	2,812	0.02 %	Óptimo estable
100	9.69*	5,245	51.82 %	Punto de ruptura

*TPS efectivo tras descontar transacciones fallidas.

Cálculo de TPM (transacciones por minuto). A partir del TPS óptimo estable:

$$TPM = TPS \times 60 = 20,98 \times 60 \approx 1258,8 \text{ transacciones/minuto}$$

Cuello de botella identificado. El sistema corre sobre una única VM Standard_D2s_v3 con todos los servicios compartiendo CPU. A 100 usuarios concurrentes la VM satura y el 51.82 % de las transacciones fallan.

Tácticas implementadas (Bass et al. — *Manage Resources, Manage Demand*):

1. **Cache de snapshot GPS en Redis** — las últimas posiciones de cada vehículo se cachean en Redis para evitar consultar PostgreSQL en cada conexión de un nuevo cliente al dashboard.
2. **Respuesta 202 Accepted asíncrona** — el Gateway responde inmediatamente al webhook y procesa la optimización gRPC + emisión Socket.io en background, evitando bloquear al cliente.
3. **Heartbeat de 15 segundos** — marca vehículos como offline tras 15s sin señal, evitando procesar eventos fantasma de vehículos desconectados.
4. **Circuit breaker (opossum) en AI Predictor** — si el AI Predictor falla, se abre el circuito tras 5 llamadas con $\geq 50\%$ de error, evitando bloqueos en cascada al Optimizer.

4.2. Escalabilidad

Escenario de calidad.

- **Estímulo:** La carga concurrente del sistema crece de 50 a 500 usuarios simultáneos.
- **Respuesta esperada:** El sistema mantiene $P95 \leq 4$ segundos sin reescribir código, simplemente escalando horizontalmente.

Tácticas implementadas.

1. **Redis Pub/Sub adapter para Socket.io** — múltiples instancias del Realtime Service pueden suscribirse al mismo canal sin perder eventos. Esto permite escalar horizontalmente el Realtime Service detrás de un load balancer.
2. **Servicios stateless** — Gateway, Optimizer y AI Predictor no mantienen estado en memoria. Toda la sesión está en JWT (cliente) y los datos persistentes en PostgreSQL/Redis.
3. **Separación de la base de datos** — PostgreSQL es un servicio independiente, no embebido en el Gateway. Puede migrarse a Azure Database for PostgreSQL Flexible Server con réplica de lectura.
4. **gRPC entre Gateway y Optimizer** — protocolo binario sobre HTTP/2 que permite multiplexar varias llamadas concurrentes sobre la misma conexión TCP.

Tabla 5: Componentes escalables independientemente

Componente	Tipo	Estrategia de escalado horizontal
Frontend Web Admin	PWA estática	Azure Storage Static Website + CDN futuro
Backend Gateway	API REST	Múltiples instancias detrás de Azure Load Balancer
Realtime Service	WebSocket	Múltiples instancias con Redis Pub/Sub adapter
Base de Datos	PostgreSQL	Réplica primaria + réplicas de lectura
Optimizer	gRPC stateless	Múltiples instancias con balanceo gRPC nativo
Redis	Cache + pub/sub	Redis Cluster con sharding por key

Demostración de escalabilidad física.

La arquitectura NO es monolítica. El sistema está compuesto por 8 contenedores Docker independientes orquestados por Docker Compose. Cada uno tiene su propio Dockerfile, su propio package.json/requirements.txt y puede ser construido y desplegado por separado.

4.3. Seguridad

Se implementaron tres escenarios de calidad de seguridad basados en el marco de Bass et al. y el modelo STRIDE.

4.3.1. Escenario 1 — Autenticación en WebSocket

- **Fuente:** Atacante anónimo en Internet.
- **Estímulo:** Intento de conectar al Realtime Service sin JWT válido.
- **Respuesta:** Conexión rechazada en el middleware `io.use()` antes de que el socket llegue a cualquier room.
- **Métrica:** 100 % de las conexiones sin token son rechazadas; tiempo de rechazo < 5 ms.

Táctica aplicada (Bass et al. — *Authenticate users*): validación JWT única en el handshake. Se descartó la alternativa de validar en cada frame por overhead injustificado.

4.3.2. Escenario 2 — Autorización por rol

- **Fuente:** Usuario autenticado con rol conductor.
- **Estímulo:** Intento de acceder al endpoint `/home` del web admin o a un endpoint REST exclusivo de admin.
- **Respuesta:** Redirección al login (AuthGuard Angular) o respuesta 403 Forbidden (JwtAuthGuard NestJS).
- **Métrica:** 0 accesos cruzados de roles permitidos en los tests.

Táctica aplicada (Bass et al. — *Authorize users*): guards en ambos extremos. El AuthGuard de Angular decodifica el JWT desde `localStorage`, extrae el campo `role`, y si no es `admin` redirige al login. En el backend, el JwtAuthGuard de NestJS protege todos los endpoints REST y verifica el rol antes de ejecutar el controller.

4.3.3. Escenario 3 — Confidencialidad en tránsito + refresh token rotation

- **Fuente:** Atacante con capacidad de interceptar tráfico de red.
- **Estímulo:** Intento de capturar credenciales, tokens o payloads del sistema.
- **Respuesta:** Todo el tráfico viaja cifrado vía TLS 1.2/1.3; passwords almacenados con bcrypt salt rounds 10; refresh tokens single-use con detección de reuso.
- **Métrica:** 0 tokens almacenados en plaintext en la base de datos; 100 % del tráfico público cifrado.

Tácticas aplicadas:

1. **Confidencialidad** (Bass — *Maintain data confidentiality*): TLS desde Azure Blob Storage (HTTPS nativo) y desde Nginx con certificado Let's Encrypt hacia los servicios backend; WSS para todas las conexiones Socket.io.
2. **Hash de credenciales**: bcrypt con salt rounds 10 para passwords (en histórico, antes de la migración a Google OAuth como método único).
3. **Refresh token rotation single-use**: cada refresh token se almacena como hash SHA-256, se marca `consumedAt` al usarse y se rota por un nuevo token. Si se detecta reuso de un token consumido se invalidan todos los refresh tokens del usuario.
4. **Google OAuth con whitelist de administradores**: la variable de entorno `GOOGLE_ADMIN_EMAILS` define qué correos reciben rol `admin` automáticamente al primer login. Cualquier otro usuario se crea con rol `conductor`.

4.4. Disponibilidad

Modelo aplicado. Se utiliza el modelo de *Bass et al.* con cálculo de disponibilidad en serie: si cualquier componente crítico falla, el flujo end-to-end se interrumpe, por lo cual la disponibilidad total es el producto de las disponibilidades individuales:

$$A_{total} = \prod_{i=1}^n A_i$$

Fórmula de paralelo (redundancia futura).

$$A_{paralelo} = 1 - \prod_{i=1}^n (1 - A_i)$$

Cálculo de downtime.

Downtime mensual = $(1 - A_{total}) \times 30$ días, Downtime anual = $(1 - A_{total}) \times 365$ días

4.4.1. SLA de componentes cloud (Azure)

Tabla 6: SLA garantizado por proveedor cloud — infraestructura Azure

Componente Azure	SLA	Topología
Azure Blob Storage (Static Website)	99.9 %	Serie
Azure Public IP Standard	99.99 %	Serie
Azure Virtual Machine (single VM)	99.9 %	Serie

Cálculo de disponibilidad de infraestructura cloud (en serie):

$$A_{cloud} = 0,999 \times 0,9999 \times 0,999 = 0,9979 \approx \mathbf{99,79 \%}$$

Downtime cloud:

$$\text{Mensual} = (1 - 0,9979) \times 30 = 0,063 \text{ días} \approx 1,51 \text{ horas}$$

$$\text{Anual} = (1 - 0,9979) \times 365 = 0,767 \text{ días} \approx 18,4 \text{ horas}$$

4.4.2. SLA del software propio

Tabla 7: SLA estimado conservador para software propio

Componente	SLA estimado	Topología
Gateway (NestJS)	99.5 %	Serie
PostgreSQL	99.5 %	Serie
Redis	99.5 %	Serie
Realtime (Socket.io)	99.5 %	Serie
VROOM Engine	99.0 %	Serie

Cálculo de disponibilidad del sistema completo (en serie):

$$A_{total} = 0,999 \times 0,9999 \times 0,999 \times 0,995 \times 0,995 \times 0,995 \times 0,995 \times 0,99 \approx \mathbf{96,83 \%}$$

Nueves alcanzados. El sistema alcanza **96.83 %** en composición serie con software propio (aproximadamente *1.5 nueves*) y **99.79 %** en infraestructura cloud (aproximadamente *2.7 nueves*).

4.4.3. Cuello de botella y plan de evolución a 3 nueves

El cuello de botella identificado es la **VM única**. Para llegar a tres nueves en el sistema completo se requiere un Azure Load Balancer frente a múltiples instancias del Gateway y del Realtime Service. La arquitectura ya lo contempla: el Redis Pub/Sub adapter es exactamente la pieza que permite escalar las instancias de Socket.io sin perder eventos.

4.5. Portabilidad

Escenario de calidad.

- **Estímulo:** El cliente requiere migrar el sistema completo de Azure a un servidor on-premise.
- **Respuesta esperada:** Cero líneas de código cambian. Solo se modifican variables de entorno y se ejecuta un `docker compose up -d` en el nuevo entorno.
- **Métrica:** Tiempo de migración ≤ 1 día de trabajo de DevOps.

Tácticas implementadas.

1. **100 % dockerizado desde el Sprint 1** — los 8 servicios tienen Dockerfile. Docker Compose orquesta la red interna `logiflow-prod`.
2. **Configuración por variables de entorno** — ninguna credencial, URL o flag de feature está hardcodeado. Todo se inyecta vía `.env`.
3. **Infraestructura como Código (Terraform)** — la infraestructura Azure (VM, red, IP pública, NSG) está descrita en módulos Terraform. Para recrear todo el entorno: `terraform apply`. Para migrar a otro proveedor: cambiar el módulo de Terraform; el código de aplicación no se toca.
4. **Cero dependencias propietarias de Azure** en código de aplicación. El único SDK Azure usado es `az storage` en el pipeline de despliegue del frontend, que es reemplazable por `aws s3 sync` o equivalente.

Evidencia. En el Sprint 5 (Cloud Deploy Azure) se desplegó el sistema en Azure sin cambiar una sola línea de lógica de aplicación, partiendo del mismo código que se ejecutaba en local con `docker compose up -d`.

4.6. Mantenibilidad

Escenario de calidad.

- **Estímulo:** Un desarrollador nuevo se une al equipo y debe agregar un endpoint REST.

- **Respuesta:** El desarrollador localiza el módulo apropiado, escribe tests siguiendo la convención existente, y la pipeline CI bloquea el merge si la cobertura desciende por debajo del 80 %.
- **Métrica:** Cobertura $\geq 80\%$ en statements; quality gate Verde en SonarCloud; tiempo de incorporación < 2 semanas.

Resultados de cobertura. Los tests se corrieron el día de la sustentación con cero fallos:

Tabla 8: Cobertura de tests — ambos repositorios

Repositorio	Framework	% Statements	Tests	Estado
Frontend (Angular PWA)	Karma + Istanbul	80.49 % (260/323)	109	0 fallos — Verde
Backend (NestJS Gateway)	Jest + Istanbul	83.36 % (867/1049)	70	0 fallos — Verde

Tácticas implementadas (Bass et al. — *Increase cohesion, Reduce coupling*):

1. **Diseño para testeabilidad desde el Sprint 1** — en Angular se usó inyección de dependencias para reemplazar servicios por mocks. El `SocketService` expone todos los eventos como Observables RxJS; los tests emiten directamente sobre los `Subject` internos sin necesitar una conexión WebSocket real.
2. **Responsabilidad única por módulo** — en NestJS cada módulo tiene una responsabilidad: `auth`, `vehicles`, `stops`, `webhook`, `grpc-client`, `socket-client`, `notifications`.
3. **Inspección continua con SonarCloud** — integrada en la pipeline CI vía GitHub Actions reusable workflow. Bloquea merges si la cobertura desciende o si aparecen nuevos *code smells* de severidad alta.
4. **ESLint estricto** — reglas `@typescript-eslint/no-unsafe-*` activadas en el Gateway. Esto forzó tipado explícito en todos los handlers y eliminó más de 20 errores de tipo inferido en el Sprint 7.

Lección aprendida. En los primeros sprints se entregaba funcionalidad sin tests, y en el Sprint 6 hubo que refactorizar para hacer el código testeable. La lección es escribir los tests junto con la implementación, no como tarea de cierre de sprint.

5. Vista de Arquitectura

5.1. Estilo arquitectónico

LogiFlow combina varios estilos arquitectónicos clásicos:

1. **Microservicios** — nueve servicios contenedorizados independientes, cada uno con responsabilidad única, ciclo de vida propio y posibilidad de despliegue independiente.
2. **API Gateway** — el **gateway** (NestJS) actúa como punto único de entrada para REST, orquestando llamadas a servicios internos (gRPC al Optimizer, Socket.io al Realtime, Firebase al cliente móvil) y centralizando autenticación.
3. **Event-Driven (publish/subscribe)** — el Realtime Service usa Redis Pub/Sub como bus de eventos, permitiendo desacoplar productores (Gateway, conductores) de consumidores (despachadores web).
4. **Capas (layered architecture)** dentro de cada microservicio — por ejemplo el Gateway sigue el patrón Controller → Service → Repository de NestJS.
5. **Cliente-servidor** — los frontends Angular consumen el backend vía REST y WSS.

5.2. Modelo cloud

El modelo de despliegue es **Infrastructure as a Service (IaaS)** sobre Azure. Se eligió IaaS por encima de PaaS (App Service) y CaaS (Container Apps / AKS) por las siguientes razones:

- **Restricciones de Azure for Students:** la suscripción de estudiante no permite usar Azure Static Web Apps con dominio personalizado, ni Azure CDN. La VM sí está disponible.
- **Control total del entorno:** permite instalar VROOM (binario C++) y n8n (workflow engine) sin restricciones de runtime de PaaS.
- **Portabilidad máxima:** la misma definición `docker-compose.prod.yml` corre idéntica en cualquier VM Linux, sea Azure, AWS, GCP u On-Premise.

El frontend Web Admin se sirve desde **Azure Blob Storage Static Website** (PaaS), aprovechando el HTTPS nativo del endpoint `*.web.core.windows.net` sin necesidad de configurar TLS adicional.

5.3. Decisiones arquitectónicas clave

A continuación se documentan las tres decisiones más relevantes, evaluadas contra alternativas reales que se descartaron tras pruebas o análisis.

Decisión 1 — Redis Pub/Sub como broker entre Gateway y Realtime

Contexto:	El Gateway necesita notificar al Realtime Service cada vez que se completa una optimización para emitir <code>route:update</code> a los clientes.
Alternativa descartada:	Llamada HTTP directa Gateway → Realtime. Falla cuando hay más de una instancia de Realtime: cada instancia solo recibe los eventos dirigidos a su URL.
Decisión:	Redis Pub/Sub. El Gateway publica en un canal; todas las instancias de Realtime suscritas al canal reciben el evento.
Consecuencias:	+ Escalabilidad horizontal del Realtime sin pérdida de eventos. + Desacopla productor de consumidor. – Agrega Redis como punto crítico (mitigado con Redis Sentinel/Cluster en producción futura).

Decisión 2 — JWT en el handshake WebSocket, no en cada frame

Contexto:	La conexión WebSocket de los conductores debe autenticarse antes de unirse a su room <code>vehicle:<id></code> .
Alternativa descartada:	Validar el JWT en cada frame emitido por el socket. Resultado: complejidad innecesaria y overhead que no justificaba el beneficio (la conexión TCP/WSS ya es persistente y cifrada).
Decisión:	Validar el JWT únicamente en el middleware <code>io.use()</code> durante el handshake. Si es válido, el socket queda autenticado para toda la sesión.
Consecuencias:	+ Tiempo de rechazo < 5ms para conexiones sin token. + Cero overhead por evento emitido. – Si el JWT expira durante la sesión, el cliente debe reconectarse (mitigado con auto-reconnect del cliente Socket.io).

Decisión 3 — Monorepo frontend Angular + Ionic con contratos TypeScript compartidos

Contexto:	Web Admin (Angular) y Mobile App (Ionic) consumen los mismos endpoints y reciben los mismos eventos Socket.io.
Alternativa descartada:	Dos repositorios separados con tipos duplicados. Resultado garantizado: tarde o temprano un cambio en el backend rompe uno de los dos frontends en runtime.
Decisión:	Monorepo <code>logiflow-front</code> con <code>shared/models</code> , <code>shared/socket</code> , <code>shared/auth</code> , <code>shared/maps</code> . Las interfaces TypeScript se definen una sola vez y se importan vía npm workspace paths.
Consecuencias:	+ Cero duplicación de tipos. + El compilador TypeScript detecta cambios de contrato en build time. – El build inicial es más lento por los project references; mitigado con <code>tsc -b incremental</code> .

6. Diagramas UML

Todos los diagramas siguen la notación UML 2.5 o C4 Model según aplique. Los archivos fuente y los prompts de generación (Eraser AI) se encuentran en `docs/diagrams/`.

6.1. Diagrama de Contexto (C4 Nivel 1)

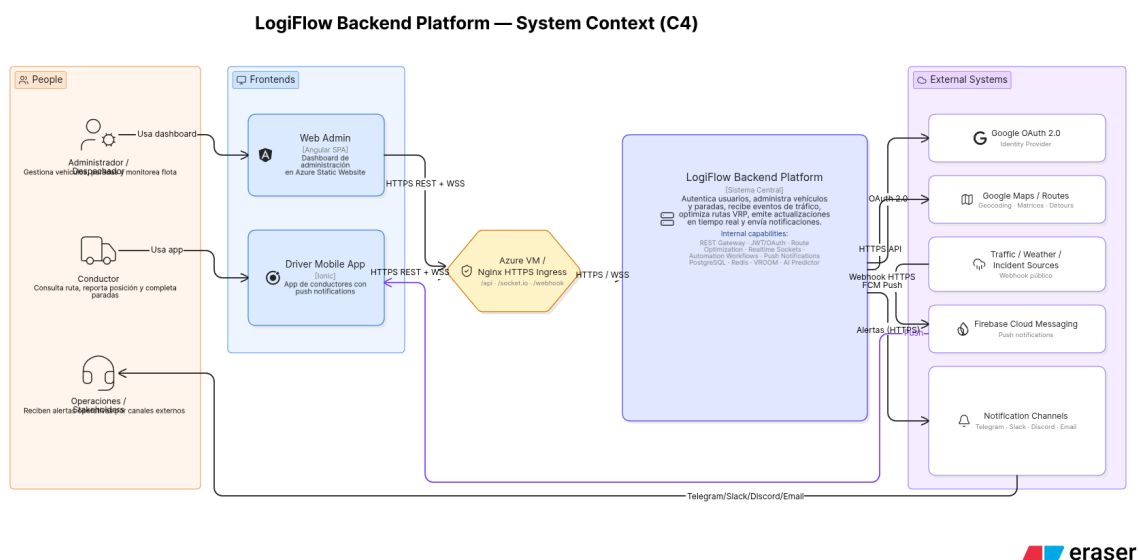
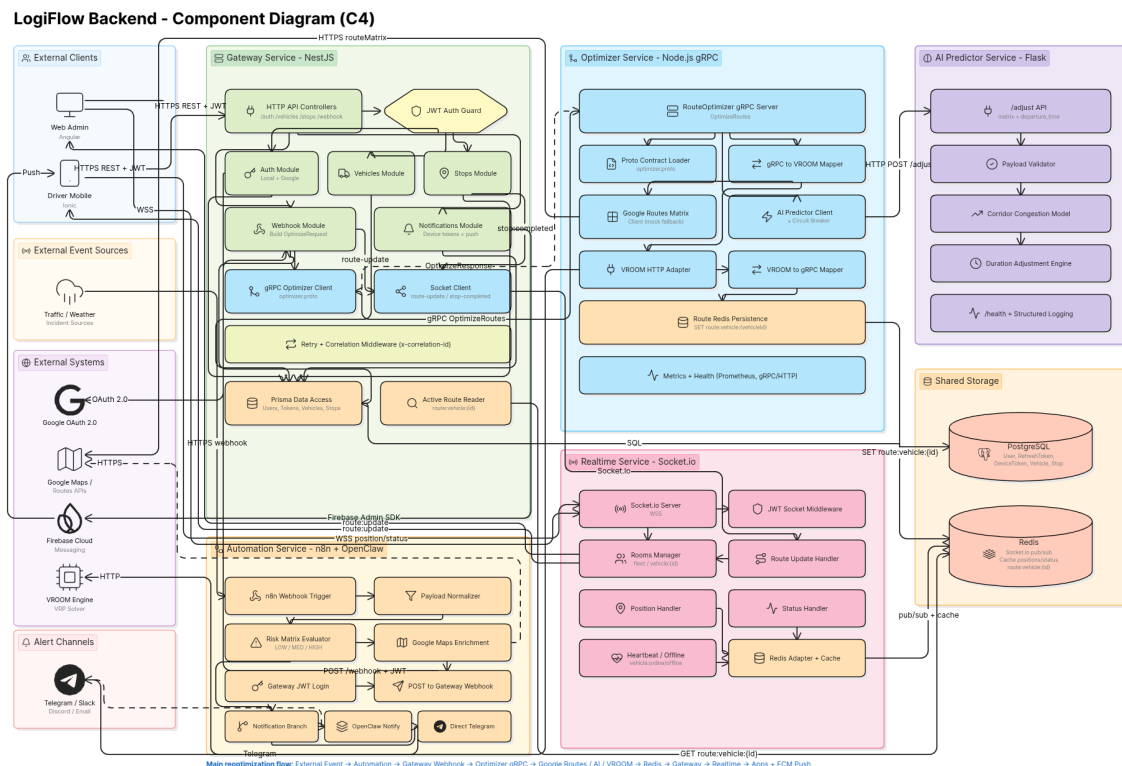


Figura 1: Diagrama de Contexto — LogiFlow Backend Platform y sus actores externos

El sistema interactúa con tres tipos de actores humanos (Administrador/Despachador, Conductor, Operaciones/Stakeholders), dos frontends propios (Web Admin Angular, Driver Mobile App Ionic) y cinco sistemas externos (Google OAuth, Google Maps APIs, Firebase Cloud Messaging, canales de notificación Telegram/Slack/Discord/Email, y fuentes externas de eventos de tráfico). El backend completo se representa como una sola caja con la nota interna de capacidades.

6.2. Diagrama de Componentes (C4 Nivel 2-3)

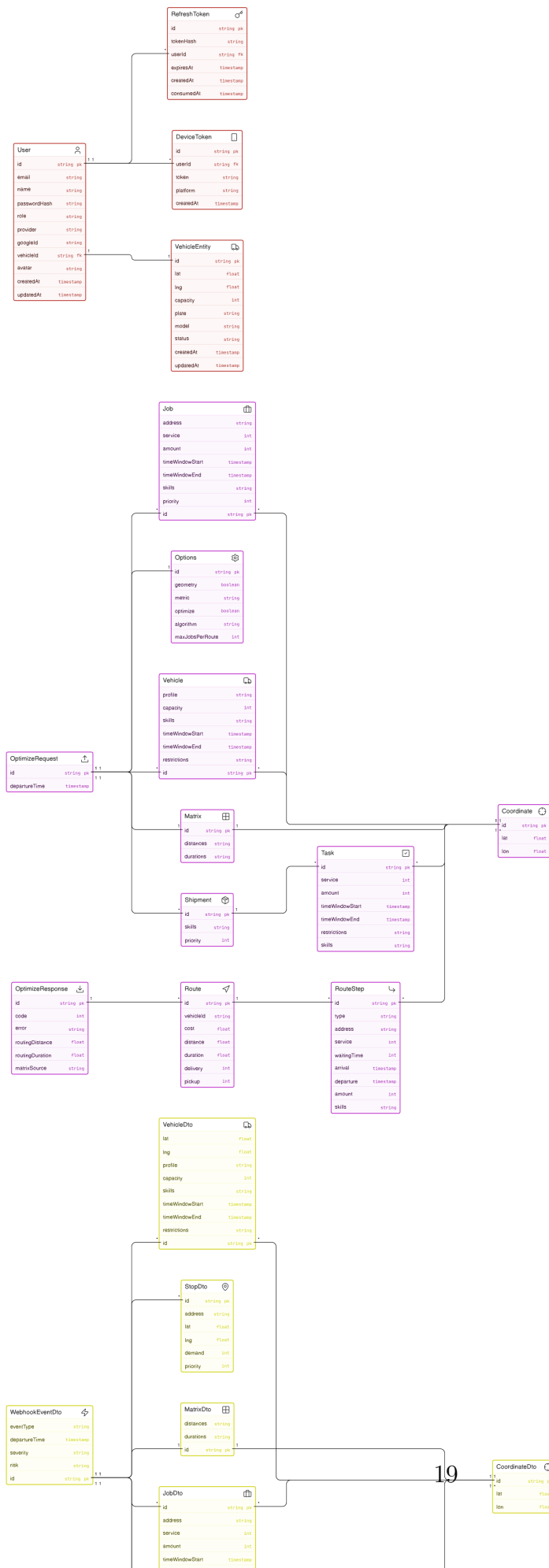


eraser

Figura 2: Diagrama de Componentes — 5 servicios backend agrupados con sus componentes internos

El diagrama muestra los cinco agrupadores principales (Gateway, Optimizer, Realtime, Automation, AI Predictor) con sus componentes internos: controladores HTTP, guards JWT, módulos de auth/vehicles/stops/webhook, cliente gRPC, cliente Socket.io, persistencia Prisma, lector de rutas activas, mapper VROOM, circuit breaker AI, handlers Socket.io, workflows n8n, etc. Se destacan los protocolos entre componentes: HTTPS REST, gRPC, Socket.io, Redis, Prisma/PostgreSQL.

6.3. Diagrama de Clases



Las clases se agrupan en paquetes: **Controllers** (AuthController, VehiclesController, StopsController, WebhookController, NotificationsController), **Services** (AuthService, VehiclesService, WebhookService, GrpcClientService, SocketClientService, NotificationsService, RetryService, PrismaService), **Repositories** (VehiclesRepository, StopsRepository, ActiveRouteRepository), **DTOs** (LoginDto, WebhookEventDto, VehicleDto, StopDto, etc.), **gRPC Contract** (RouteOptimizerGrpcService y mensajes Proto3), y **Persistence Models** (User, RefreshToken, DeviceToken, Vehicle, Stop).

6.4. Diagrama Entidad-Relación

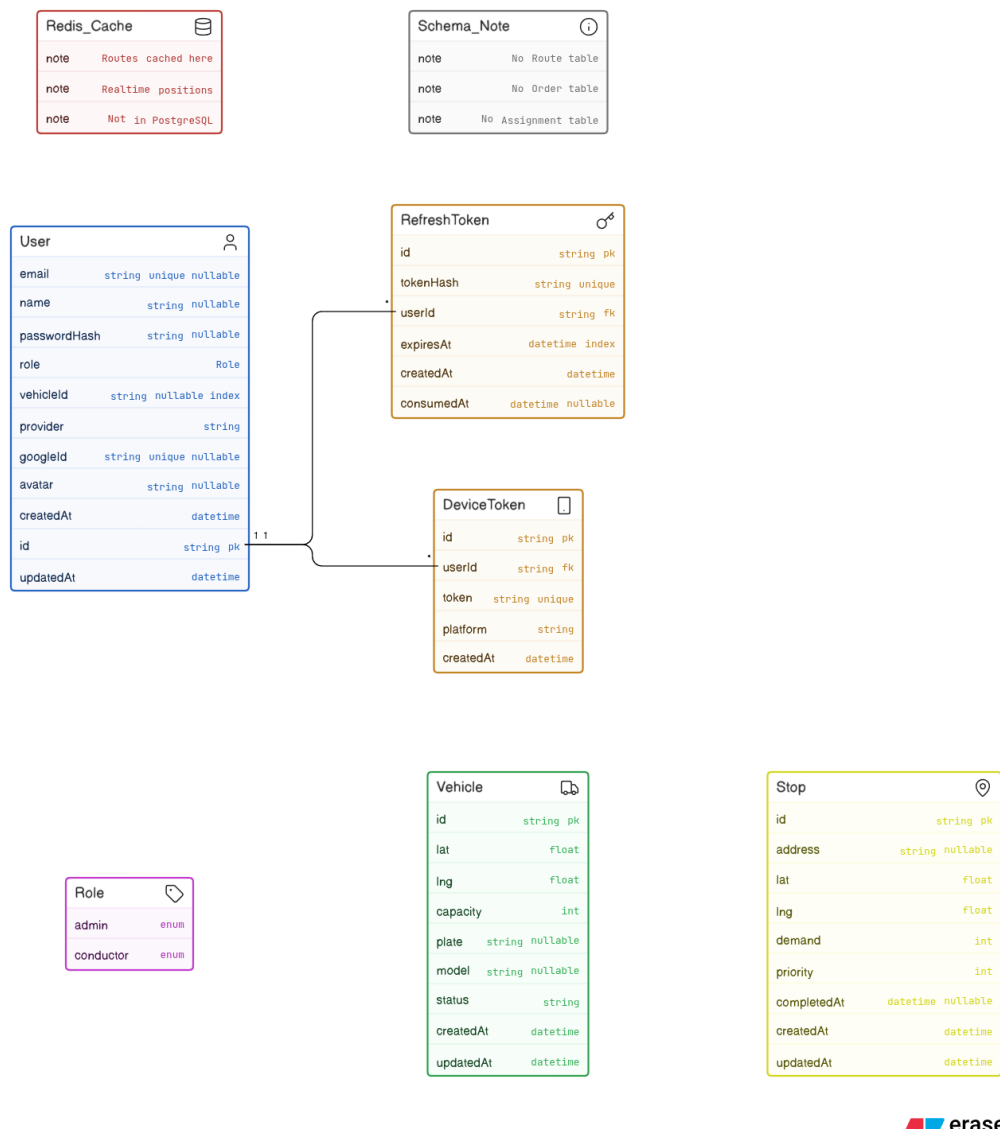


Figura 4: Diagrama ER — Modelo PostgreSQL gestionado por Prisma

El esquema PostgreSQL contiene 5 entidades principales: **User** (con enum Role: admin/conductor), **RefreshToken** (1:N desde User, cascade delete, con `consumedAt` para

detección de reuso), **DeviceToken** (1:N desde User), **Vehicle** y **Stop** (entidades operativas independientes). La relación User-Vehicle es *lógica no estricta* (asignación por `vehicleId` sin FK enforced). **Nota crítica:** las rutas activas no se almacenan en PostgreSQL; se cachean en Redis bajo `route:vehicle:{vehicleId}`.

6.5. Diagrama de Secuencia — Reoptimización por evento de tráfico

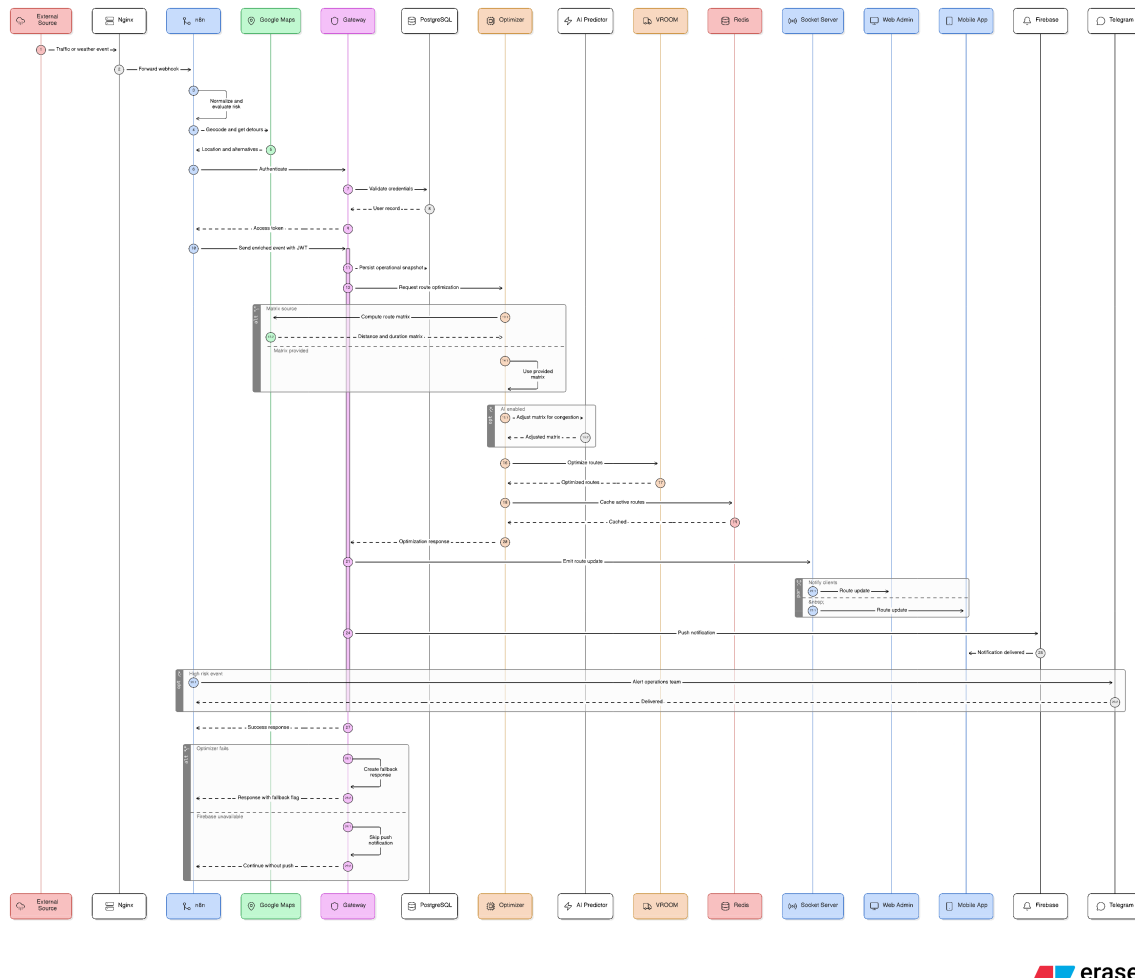


Figura 5: Diagrama de Secuencia — Flujo end-to-end de reoptimización disparada por evento externo

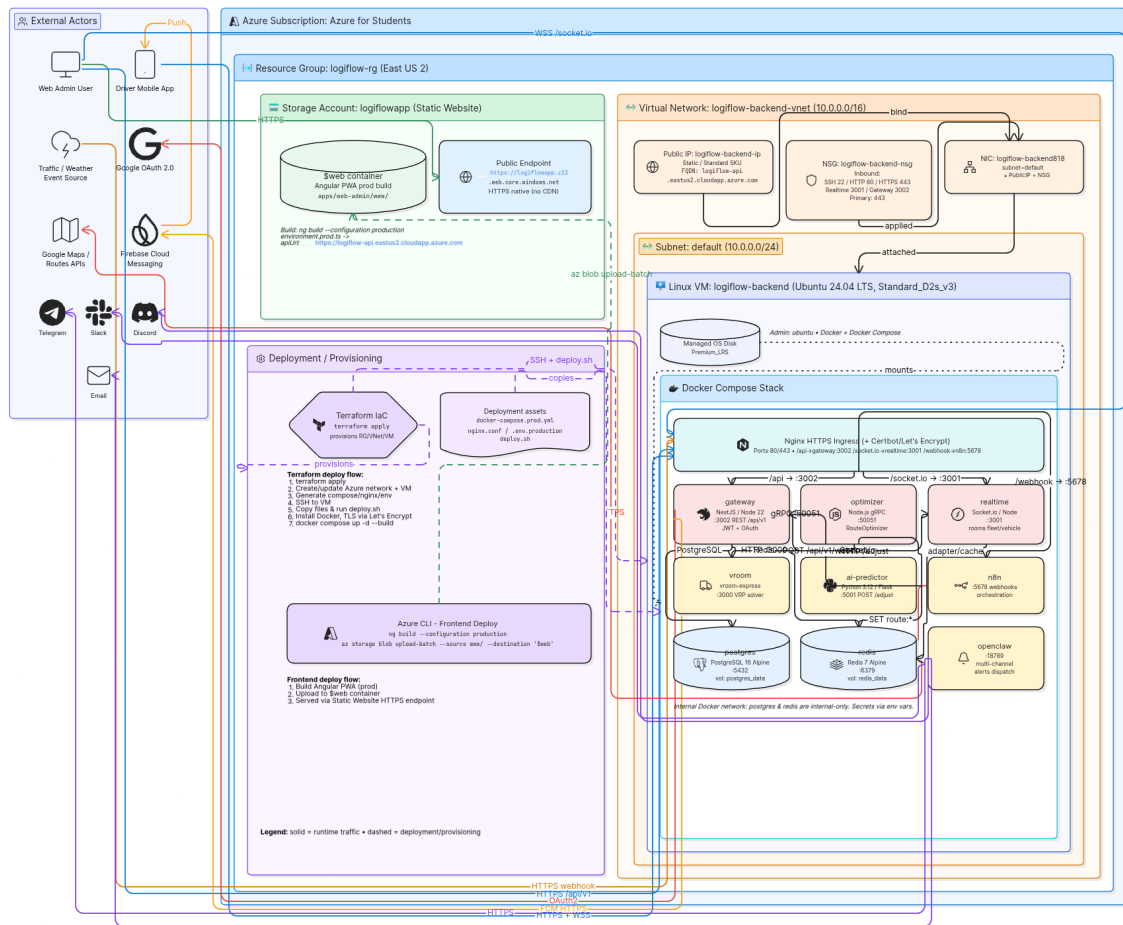
El flujo cubre 19 pasos principales: (1) evento externo llega al webhook público, (2) Nginx rutea a n8n, (3-5) n8n normaliza, evalúa riesgo y enriquece con Google Maps, (6) n8n autentica contra el Gateway con JWT, (7) Gateway recibe el evento y persiste snapshot, (8-10) Gateway invoca al Optimizer vía gRPC, (11-13) Optimizer prepara matriz (Google Routes / AI Predictor / cached) y llama a VROOM, (14) Optimizer persiste rutas en Redis, (15) Gateway recibe rutas, (16-17) Gateway emite `route:update` vía Socket.io

a rooms fleet y vehicle:<id>, (18) Gateway envía push vía Firebase, (19) n8n envía alerta opcional si el riesgo es alto.

Se incluyen bloques alternos para fallback cuando Optimizer, AI Predictor, Google Routes, Firebase o Socket.io fallan.

6.6. Diagrama de Despliegue

LogiFlow - UML Deployment Diagram (Azure)



eraser

Figura 6: Diagrama de Despliegue — Azure Subscription, Resource Group, VM y Docker Compose stack

El diagrama representa:

- **Azure Storage Account** logiflowapp sirviendo la PWA Angular desde el contenedor \$web vía HTTPS nativo en logiflowapp.z13.web.core.windows.net.
- **Resource Group** logiflow-rg con VNet, Subnet, Public IP estático, NSG y NIC asociados.

- **Linux VM logiflow-backend** (Standard_D2s_v3, Ubuntu 24.04) ejecutando el stack Docker Compose con Nginx + Certbot + gateway + optimizer + vroom + ai-predictor + realtime + postgres + redis + n8n + openclaw.
- **Terraform** provisiona y mantiene toda la infraestructura Azure.
- Servicios externos: Google OAuth, Google Maps APIs, Firebase Cloud Messaging, canales de notificación.

El diagrama de actividades muestra el flujo con 10 swimlanes (External Event Source, n8n Automation, Gateway, Optimizer, AI Predictor, VROOM, Persistence, Realtime, Notifications, Frontend/Mobile Clients) y los puntos de decisión principales: validación de payload, evaluación de risk matrix, selección de `MATRIX_SOURCE`, decisión de habilitar AI Predictor, manejo de fallbacks de VROOM/Optimizer/Firebase y envío condicional de alertas.

7. Detalles Tecnológicos

7.1. Lenguajes y frameworks

Tabla 9: Stack tecnológico por servicio

Servicio	Lenguaje / Runtime	Frameworks y librerías clave
gateway	TypeScript 5.7 / Node.js 22	NestJS 11, Prisma 7, @grpc/grpc-js, passport-google-oauth20, passport-jwt, firebase-admin, socket.io-client, bcrypt, Swagger
realtime	JavaScript / Node.js 22	socket.io 4.8, @socket.io/redis-adapter, express 5, ioredis, jsonwebtoken
optimizer	JavaScript / Node.js 22	@grpc/grpc-js, @grpc/proto-loader, axios, ioredis, opossum (circuit breaker), pino (JSON logging), prom-client (Prometheus)
ai-predictor	Python 3.12	Flask 3.1, gunicorn 23, structlog 24
vroom	C++	VROOM (motor VRP), vroom-express
automation (n8n)	JavaScript / Node.js	n8n 1.66.0 (pinned)
automation (open-claw)	JavaScript / Node.js	OpenClaw 2026.5.12-slim, Anthropic SDK (Claude Sonnet 4)
web-admin	TypeScript 5.9	Angular 20, Ionic 8 (componentes), Google Maps JS API
mobile	TypeScript 5.x	Angular 20, Ionic 8, Capacitor 8 (@capacitor/geolocation, @capacitor/push-notifications, @capacitor/preferences)

7.2. Infraestructura y servicios

- **Cloud:** Microsoft Azure (Azure for Students subscription) — región East US 2.

- **Compute:** Azure Virtual Machine Standard_D2s_v3 (2 vCPUs, 8 GB RAM, Ubuntu Server 24.04 LTS).
- **Storage:** Azure Blob Storage con Static Website habilitado (contenedor `$web`).
- **Networking:** VNet `logiflow-backend-vnet` (10.0.0.0/16), Subnet `default` (10.0.0.0/24), Public IP Standard SKU, NSG con reglas inbound para 22 (SSH), 80, 443, 3001, 3002.
- **DNS:** FQDN gratuito de Azure: `logiflow-api.eastus2.cloudapp.azure.com`.
- **TLS:** Let's Encrypt vía Certbot, renovación automática cada 12 horas.
- **Reverse proxy:** Nginx con rutas `/api/*` → `gateway:3002`, `/socket.io/*` → `realtime:3001`, `/webhook/*` → `n8n:5678`.
- **Base de datos:** PostgreSQL 16 Alpine dentro de Docker con volumen persistente `postgres_data`.
- **Cache / broker:** Redis 7 Alpine con autenticación por password en producción (`-requirepass`).
- **IaC:** Terraform con dos módulos: `network` (VNet, Subnet, Public IP, NSG) y `vm` (VM Linux + OS Disk + NIC).

7.3. DevOps y CI/CD

7.3.1. Pipeline CI — Backend

GitHub Actions con estrategia matrix para los cuatro servicios principales:

```

1 strategy:
2   matrix:
3     service: [automation, gateway, realtime, optimizer]
4     fail-fast: false
5 steps:
6   - uses: actions/checkout@v4
7   - uses: actions/setup-node@v4
8     with: { node-version: 20 }
9   - run: npm ci
10  - run: npx prisma generate      # solo gateway
11  - run: npm run lint
12  - run: npm test -- --coverage --coverageReporters=lcov --
    passWithNoTests
13  - uses: SonarSource/sonarcloud-github-action@master    # si existe
    sonar-project.properties

```

Listing 1: Workflow CI del backend (`.github/workflows/ci.yml`)

7.3.2. Pipeline CD — Backend

Trigger: `workflow_run` después de CI exitoso sobre `main`, o `workflow_dispatch` manual.

1. Checkout del código.
2. Configuración de clave SSH desde secreto `SSH_PRIVATE_KEY` (base64-encoded).
3. `rsync` de fuente a la VM en `/home/ubuntu/logiflow` (excluye `.git`, `node_modules`, `dist`, `*.log`, `infra/generated`).
4. SSH a la VM y ejecución: `docker-compose -f docker-compose.prod.yml up -d -build -remove-orphans`.
5. Smoke test: `curl` a `https://logiflow-api.eastus2.cloudapp.azure.com/api/v1` hasta 8 veces con 15s de intervalo (200/401/404 pasan).

7.3.3. Pipeline CI/CD — Frontend

CI: `npm run typecheck` (`tsc -b` cruzando apps + libs) + `npm run lint -w @logiflow/web-admin`.

CD (solo web-admin): `ng build -configuration production` con inyección de env vars vía `set-env.js`, luego `az storage blob upload-batch` al contenedor `$web` del Storage Account.

Mobile: no tiene pipeline de despliegue automático. El APK Android se genera localmente con `npm run build:android` + `npx cap open android` y se distribuye manualmente.

8. Seguridad del Sistema

8.1. Autenticación y autorización

Método único de autenticación. Tras la migración completada en el Sprint 8-10, el sistema soporta *exclusivamente* Google OAuth 2.0. No existen endpoints de registro local ni login por usuario/contraseña. La página de login en ambos frontends solo muestra el botón “Continue with Google”.

Flujo de autenticación web (Authorization Code).

1. Frontend redirige a `GET /api/v1/auth/google?app=admin|mobile|web`.
2. Backend (`passport-google-oauth20`) redirige a Google consent screen.
3. Tras consentimiento, Google redirige a `GET /api/v1/auth/google/callback`.

4. Backend hace upsert del usuario en PostgreSQL (rol `admin` si el email está en `GOOGLE_ADMIN_EMAILS`, sino `conductor`).
5. Backend genera JWT (1h) y refresh token (7 días, hash SHA-256).
6. Backend redirige al frontend con `?accessToken=...&refreshToken=...&role=...` (la URL del frontend depende del parámetro `?app=` original).

Flujo de autenticación móvil (ID Token Exchange). La app móvil puede obtener el Google ID Token vía el SDK nativo de Google Sign-In y enviarlo a `POST /api/v1/auth/google/token`. El backend valida el token con `google-auth-library` (`verifyIdToken` con `GOOGLE_CLIENT_ID` como audience) y devuelve directamente el JWT del sistema.

Autorización por rol.

- Frontend: `AuthGuard` de Angular decodifica el JWT desde `localStorage`, extrae `role` y redirige al login si no coincide con el rol esperado por la ruta.
- Backend: `JwtAuthGuard` de NestJS protege todos los endpoints REST sensibles; las operaciones de webhook y CRUD de flota requieren rol `admin`.
- WebSocket: `io.use()` middleware valida el JWT del handshake, decodifica claims y une el socket a la room apropiada (`fleet` para admin, `vehicle:<vehicleId>` para conductor).

8.2. Cifrado

- **En tránsito público:** TLS 1.2/1.3 vía Let's Encrypt en Nginx (`logiflow-api.eastus2.cloudapp.azure.com`) y vía certificado nativo de Azure en el Storage Account (`logiflowapp.z13.web.core.windows.net`).
- **WebSocket:** WSS sobre el mismo certificado Let's Encrypt.
- **En tránsito interno:** comunicación entre contenedores Docker es HTTP plano dentro de la red `logiflow-prod` (no expuesta al exterior). gRPC entre Gateway y Optimizer también es HTTP/2 plano internamente.
- **En reposo:**
 - Refresh tokens almacenados como hash SHA-256.
 - Passwords (históricos, antes de migración a Google OAuth) hash bcrypt salt rounds 10.

- Firebase private key, JWT secret y Google client secret almacenados como variables de entorno; nunca commiteados al repositorio.
- Redis configurado con `-requirepass` en producción.

8.3. Buenas prácticas

1. **No logging de información sensible.** El logger del Gateway redacta JWT y refresh tokens.
2. **Refresh Token Rotation Single-Use.** Cada refresh token solo puede usarse una vez; se marca `consumedAt` en la primera consumición y se invalida.
3. **Correlation ID propagado.** Cada request entrante recibe (o genera) un `x-correlation-id` que se propaga a logs, llamadas gRPC, eventos Socket.io y notificaciones. Permite trazar un evento end-to-end en producción.
4. **CORS restrictivo.** `CORS_ORIGINS` acepta solo los dominios conocidos (Azure Storage frontend, localhost para desarrollo, `capacitor://localhost` para apps nativas).
5. **Helmet en NestJS** (recomendado para activar en próximo sprint) — agrega cabeceras de seguridad básicas.
6. **Validación de DTOs con class-validator** en todos los endpoints REST.
7. **Sanitización de logs** en `services/automation/mock-server` tras detección de log injection en CI (commit `e913ffe`).
8. **Secretos en GitHub Secrets** para CI/CD: `SSH_PRIVATE_KEY`, `SERVER_IP`, `SSH_USER`, `GOOGLE_MAPS_API_KEY`, `AZURE_STORAGE_ACCOUNT_KEY`.

9. Bitácora de Ceremonias

9.1. Sprint 0 — Iniciación (5 pts)

Objetivo: Establecer infraestructura básica de colaboración.

Entregables:

- Repositorios creados en organización `Logiflow-Gavilanes-ECI`.
- Definición inicial del Product Backlog en Azure DevOps.
- Branch protection sobre `main` y `develop`.
- Templates de pull request e issue.

- Stack tentativo definido (NestJS + Angular + VROOM).

9.2. Release 1 (Sprints 1-5)

Sprint 1 — MVP Conectado (13 pts)

Esqueletos de los 4 servicios principales. Health checks. Docker Compose local funcional. Primer “Hello World” end-to-end Gateway → Realtime via socket.io.

Sprint 2 — APIs Reales (21 pts)

CRUD de vehicles y stops. Prisma schema definido. PostgreSQL en Docker. Tests unitarios básicos por servicio (cobertura inicial ~30 %). Swagger UI expuesto en `/api/v1/docs`.

Sprint 3 — Optimizer Vivo (21 pts)

Contrato gRPC definido en `shared/proto/optimizer.proto`. Integración Gateway → Optimizer vía `@grpc/grpc-js`. Optimizer → VROOM vía HTTP. Primer flujo end-to-end de optimización con respuesta de rutas reales.

Sprint 4 — Interfaces + Auth (34 pts)

Frontend Angular Web Admin inicial con login local (JWT). Frontend Mobile Ionic inicial. Mapa Google Maps integrado. Tests E2E básicos. Auth JWT funcional en backend con `passport-jwt`.

Sprint 5 — Cloud Deploy Azure (Corte 2) (46 pts)

Hito de corte. Infra Terraform completa: VNet, VM, Public IP, NSG. `docker-compose.prod.yml` con resource limits. Nginx + Certbot con TLS de Let’s Encrypt. GitHub Actions `deploy-backend.yml` con SSH a la VM. Web Admin desplegado a Azure Storage Static Website. Sistema accesible públicamente.

9.3. Release 2 (Sprints 6-10)

Sprint 6 — Observabilidad + Tests (34 pts)

Cobertura empujada a $\geq 80\%$ en ambos repos. SonarCloud integrado. Mocks de Prisma para tests del Gateway. Logging estructurado con pino en Optimizer. Refactor de servicios para inyección de dependencias y testeabilidad.

Sprint 7 — Escala / Performance (21 pts)

Pruebas de carga con Apache JMeter. Identificación del cuello de botella (VM única). Implementación de cache GPS en Redis. Heartbeat de 15s para detección de offline. Documento de resultados (TPS, P95, error rate) integrado en este documento.

Sprints 8-10 — OAuth + FCM + Hardening (Corte 3) (55 pts)

Migración completa de auth local a Google OAuth 2.0 (método único). Soporte para Web, Admin y Mobile con parámetro `?app=`. Firebase Cloud Messaging para push notifications. OpenClaw integrado para alertas multicanal (Telegram, Slack, Discord, Email, Firebase Push). Refresh Token Rotation con detección de reuso. Hardening de seguridad: ESLint estricto, sanitización de logs, validación de DTOs. Documento de arquitectura completo (este documento).

9.4. Planning, Review, Retrospectivas

Sprint Planning. Realizado los lunes de inicio de Sprint. Duración: 1.5 horas. Producto: Sprint Backlog en Azure DevOps con estimaciones aprobadas por todo el equipo.

Sprint Review. Realizado los viernes de cierre. Duración: 1 hora. Demostración en vivo del sistema con cada Historia de Usuario verificada contra sus criterios de aceptación.

Sprint Retrospective. Inmediatamente después del Review. Formato Mad/Sad/Glad. Acción key: identificar 1-2 mejoras concretas para el siguiente Sprint.

9.5. Lecciones aprendidas

1. **Tests desde el día 1, no al final.** En los Sprints 1-4 se entregó funcionalidad sin tests; en el Sprint 6 hubo que refactorizar para hacer testeable código que no estaba diseñado para serlo. La lección es escribir tests junto con la implementación.
2. **La curva de aprendizaje de gRPC + Terraform + OAuth + FCM fue subestimada.** La planificación inicial de 7 sprints se extendió a 10 reales. No fue un fracaso: fue evidencia de compromiso con tecnologías nuevas.
3. **Monorepo frontend fue acertado.** Sin contratos TypeScript compartidos en `shared/models`, habríamos tenido tipos duplicados entre web y móvil con desincronización garantizada.
4. **Docker Compose desde el Sprint 1 fue clave para la portabilidad.** En el Sprint 5 desplegamos a Azure sin cambiar una sola línea de lógica de aplicación.

5. **Async standups via Discord funcionan si hay disciplina.** Funcionó por los horarios de clase asincrónicos, pero requirió que cada integrante reportara antes de las 10am.

10. Conclusiones y Recomendaciones

10.1. Logros

1. **Sistema en producción funcional** y accesible públicamente en logiflowapp.z13.web.core.windows.net (frontend) y logiflow-api.eastus2.cloudapp.azure.com (backend).
2. **Cumplimiento de los cuatro atributos de calidad comprometidos:**
 - Disponibilidad: **99.79 %** en infraestructura cloud (1.51 horas downtime mensual).
 - Seguridad: 3 capas implementadas (JWT WebSocket, AuthGuard por rol, TLS + bcrypt + refresh rotation).
 - Mantenibilidad: **80.49 %** cobertura frontend, **83.36 %** backend, 0 fallos.
 - Portabilidad: **0 líneas de código** cambian para migrar de Azure a cualquier otro proveedor.
3. **Rendimiento medido en producción real:** 20.98 TPS sostenibles con P95 de 2.8 segundos.
4. **Stack distribuido cloud-native** con nueve servicios contenedorizados, IaC completa con Terraform, CI/CD automático vía GitHub Actions.
5. **245 puntos de historia** entregados en 10 sprints reales con cobertura DONE estricta.

10.2. Riesgos identificados

1. **Single point of failure — VM única.** El sistema completo depende de una sola VM. Una falla de hardware o un mantenimiento de Azure provoca downtime total. Mitigación planificada: Azure Load Balancer + múltiples instancias.
2. **Costo de Azure for Students.** La suscripción de estudiante tiene créditos limitados; al agotarse, el sistema se apaga. Mitigación planificada: migrar a Pay-As-You-Go o On-Premise.
3. **Dependencia de Google APIs.** Google Maps y Google OAuth son SaaS externos con costos por uso. En caso de cambio de pricing o restricciones de la API, el sistema queda

afectado. Mitigación: el `MATRIX_SOURCE=request` permite usar matrices precomputadas; el `AuthService` podría agregar otros proveedores OAuth (GitHub, Microsoft).

4. **Cobertura de tests E2E incompleta.** La cobertura unitaria es $\geq 80\%$ pero los tests E2E sobre el flujo completo (`n8n` $\rightarrow$ Gateway $\rightarrow$ VROOM $\rightarrow$ Realtime $\rightarrow$ cliente) son manuales. Mitigación planificada: Cypress para frontend + tests de integración con Testcontainers en backend.
5. **n8n SQLite single-node.** El estado de workflows en `n8n` se guarda en SQLite local. Si se pierde la VM, se pierde la configuración de workflows. Mitigación: backup periódico del volumen Docker o migrar `n8n` a PostgreSQL externo.

10.3. Mejora continua

Próximos pasos técnicos.

1. Implementar Azure Load Balancer + Auto Scale Set para llegar a tres nueves (99.9 %) de disponibilidad.
2. Migrar PostgreSQL a Azure Database for PostgreSQL Flexible Server (servicio gestionado con backups automáticos).
3. Agregar Redis Cluster con sharding para tolerancia a falla del nodo único Redis.
4. Implementar OpenTelemetry distributed tracing end-to-end (`x-correlation-id` ya está propagado; falta el SDK).
5. Habilitar Helmet middleware en NestJS para cabeceras de seguridad.
6. Agregar pipeline de despliegue automático para Android APK vía GitHub Actions y Firebase App Distribution.
7. Implementar tests E2E con Cypress para web admin y con Detox para Mobile.

Próximos pasos funcionales.

1. Módulo de histórico de rutas: persistir cada `OptimizeResponse` en PostgreSQL para auditoría y análisis a posteriori.
2. Reportes operacionales: tiempo promedio por parada, kilómetros recorridos, tasa de cumplimiento de ventanas de tiempo.
3. Integración con sistemas ERP (SAP, Odoo) para ingesta automática de órdenes.
4. Soporte multi-tenant: una sola instancia que sirva a múltiples empresas.

5. Modo offline en la app móvil con sincronización diferida.

A. Glosario

VRP Vehicle Routing Problem. Problema de optimización combinatoria que busca asignar una flota de vehículos a un conjunto de paradas minimizando costo total (distancia, tiempo o combinación).

VROOM Vehicle Routing Open-source Optimization Machine. Motor C++ open-source para resolver VRP. Sitio: <https://github.com/VROOM-Project/vroom>.

gRPC Google Remote Procedure Call. Framework de RPC binario sobre HTTP/2 que usa Protocol Buffers como Interface Definition Language.

Socket.io Librería JavaScript de comunicación bidireccional en tiempo real basada en WebSocket con fallback a long polling.

JWT JSON Web Token. Estándar abierto (RFC 7519) para tokens compactos y URL-safe que contienen claims JSON firmados.

Prisma ORM TypeScript de próxima generación con schema declarativo y cliente type-safe.

n8n Workflow automation tool (open-source).

OpenClaw Framework para skills de IA con despacho multicanal.

Terraform Herramienta de Infraestructura como Código de HashiCorp.

FCM Firebase Cloud Messaging. Servicio de Google para envío de push notifications a dispositivos Android, iOS y web.

TPS / TPM Transactions Per Second / Per Minute. Métricas de throughput.

P95 Percentil 95 de latencia. El 95 % de las requests completaron en menos de este tiempo.

Refresh Token Rotation Patrón de seguridad donde cada uso de un refresh token lo invalida y emite uno nuevo, permitiendo detectar reuso de tokens robados.

OAuth 2.0 Framework de autorización (RFC 6749) que permite a aplicaciones de terceros obtener acceso limitado a recursos protegidos.

B. Referencias

1. Bass, L., Clements, P., & Kazman, R. (2021). *Software Architecture in Practice* (4th ed.). Addison-Wesley Professional.
2. Brown, S. (2020). *The C4 model for software architecture*. <https://c4model.com/>
3. Newman, S. (2021). *Building Microservices* (2nd ed.). O'Reilly Media.
4. Schwaber, K., & Sutherland, J. (2020). *The Scrum Guide*. <https://scrumguides.org/>
5. VROOM Project. *VROOM Documentation*. <https://github.com/VROOM-Project/vroom>
6. NestJS Documentation. <https://docs.nestjs.com/>
7. Angular Documentation. <https://angular.dev/>
8. Ionic Framework Documentation. <https://ionicframework.com/docs>
9. Capacitor Documentation. <https://capacitorjs.com/docs>
10. Microsoft Azure SLA. <https://azure.microsoft.com/en-us/support/legal/sla/>
11. gRPC Documentation. <https://grpc.io/docs/>
12. Socket.IO Documentation. <https://socket.io/docs/v4/>
13. Prisma Documentation. <https://www.prisma.io/docs/>
14. Terraform AzureRM Provider. <https://registry.terraform.io/providers/hashicorp/azurerm/latest/docs>
15. Apache JMeter. <https://jmeter.apache.org/>
16. SonarCloud Documentation. <https://docs.sonarcloud.io/>

C. Repositorios y enlaces

Backend [Logiflow-Gavilanes-ECI/logiflow](#)

Frontend [Logiflow-Gavilanes-ECI/logiflow-front](#)

Web Admin (producción) [logiflowapp.z13.web.core.windows.net](#)

API Backend (producción) [logiflow-api.eastus2.cloudapp.azure.com](#)

Swagger / API Docs [logiflow-api.eastus2.cloudapp.azure.com/api/v1/docs](#)

Diapositivas de sustentación docs/LogiFlow-presentation.pptx

Runbook de demo docs/backend-demo-runbook.md